```
"""
Team scanner: 120+ virtual analysts (rule-based) har coin par vote karte hain.
sol_bot.py ke saath ek hi folder me rakho.

Har analyst alag strategy + timeframe (15m / 1h / 4h) hai:
EMA crossovers, price vs EMA, RSI, SuperTrend, MACD, Donchian breakout,
momentum (ROC) + funding rate + long/short ratio.
Vote: +1 bullish, -1 bearish, 0 neutral.
Team ka consensus % se signal banta hai.

ENV: TG_TOKEN, TG_CHAT_ID, optional TOP_N (default 30), CONSENSUS (default 0.55)
Sirf alerts, advice nahi.
"""
import os
import time

import sol_bot as bot

TOP_N = int(os.getenv("TOP_N", "30"))
CONSENSUS = float(os.getenv("CONSENSUS", "0.55"))  # total analysts ka itna % ek taraf ho
MAX_IN_MSG = 8
SKIP = ("3L", "3S", "5L", "5S", "UP_", "DOWN_", "BULL", "BEAR")
```

```
TFS = ["15m", "1h", "4h"]


def top_pairs(n):
    rows = []
    for t in bot.get("/spot/tickers"):
        p = t["currency_pair"]
        if not p.endswith("_USDT") or any(s
in p for s in SKIP):
            continue
        try:

rows.append((float(t["quote_volume"]), p))
        except (KeyError, ValueError,
TypeError):
            continue
    rows.sort(reverse=True)
    return [p for _, p in rows[:n]]


def sign(x):
    return 1 if x > 0 else -1 if x < 0 else
0


def tf_votes(h, l, c):
    """Ek timeframe ke sare analysts ke
votes (list of +1/-1/0)."""
    votes = []
    n = len(c)

    # 1) EMA crossover analysts
    ema_cache = {}
```

```
    def E(p):
        if p not in ema_cache:
            ema_cache[p] = bot.ema(c, p)
        return ema_cache[p]

    for fast in (3, 5, 8, 10, 13, 21):
        for slow in (20, 30, 50, 100):
            if fast < slow and n > slow *
2:
                votes.append(sign(E(fast)
[-1] - E(slow)[-1]))

    # 2) Price vs EMA
    for p in (50, 100, 200):
        if n > p * 1.2:
            votes.append(sign(c[-1] - E(p)
[-1]))

    # 3) RSI analysts
    for p in (7, 14, 21):
        if n > p * 3:
            r = bot.rsi(c, p)
            votes.append(1 if r > 55 else
-1 if r < 45 else 0)

    # 4) SuperTrend analysts
    for per, mult in ((7, 2.0), (10, 3.0),
(14, 3.0), (20, 4.0)):
        if n > per * 3:
            _, tr = bot.supertrend(h, l, c,
per, mult)
            votes.append(tr)

    # 5) MACD analysts
```

```
    for f, s, sg in ((12, 26, 9), (5, 13,
5)):
        if n > s * 3:
            macd = [a - b for a, b in
zip(bot.ema(c, f), bot.ema(c, s))]
            sig = bot.ema(macd, sg)
            votes.append(sign(macd[-1] -
sig[-1]))

    # 6) Donchian breakout analysts
    for p in (10, 20, 50):
        if n > p + 5:
            hh, ll = max(h[-p - 1:-1]),
min(l[-p - 1:-1])
            votes.append(1 if c[-1] > hh
else -1 if c[-1] < ll else 0)

    # 7) Momentum (ROC) analysts
    for p in (10, 20):
        if n > p + 5:
            votes.append(sign(c[-1] - c[-1
- p]))

    return votes


def analyse_team(pair):
    bot.PAIR = pair
    votes = []
    h1 = l1 = c1 = None
    for tf in TFS:
        h, l, c = bot.candles(tf, 300)
        if len(c) < 60:
            raise ValueError(f"{tf} data
```

```
kam hai")
        votes += tf_votes(h, l, c)
        if tf == "1h":
            h1, l1, c1 = h, l, c

    # Derivatives analysts (contrarian)
    fr = bot.funding_rate()
    if fr is not None:
        votes.append(-1 if fr > 0.0005 else
1 if fr < -0.0005 else 0)
    lsr = bot.long_short_ratio()
    if lsr is not None:
        votes.append(-1 if lsr > 1.5 else 1
if lsr < 0.8 else 0)

    total = len(votes)
    bull, bear = votes.count(1),
votes.count(-1)
    price = c1[-1]
    a = bot.atr(h1, l1, c1)[-1]
    risk = bot.ATR_SL_MULT * a

    if bull / total >= CONSENSUS:
        bias = "LONG"
        sl, t1, t2 = price - risk, price +
bot.T1_R * risk, price + bot.T2_R * risk
    elif bear / total >= CONSENSUS:
        bias = "SHORT"
        sl, t1, t2 = price + risk, price -
bot.T1_R * risk, price - bot.T2_R * risk
    else:
        bias, sl, t1, t2 = "WAIT", None,
None, None
```

```
    return dict(pair=pair, price=price,
bias=bias, bull=bull, bear=bear,
                total=total, sl=sl, t1=t1,
t2=t2)


def scan():
    out = []
    for pair in top_pairs(TOP_N):
        try:
            x = analyse_team(pair)
        except Exception as e:
            print(f"{pair}: skip ({e})")
            continue
        out.append(x)
        print(f"{pair}: {x['bias']} bull
{x['bull']} bear {x['bear']} /
{x['total']}")
        time.sleep(0.3)
    return out


def fmt(x):
    d = 4 if x["price"] < 10 else 2
    pct = (x["bull"] if x["bias"] == "LONG"
else x["bear"]) / x["total"] * 100
    return (f"{x['pair']} {x['bias']} -
{pct:.0f}% team agree "
            f"({x['bull']} bull /
{x['bear']} bear / {x['total']})\n"
            f"Entry ~{x['price']:.{d}f} |
SL {x['sl']:.{d}f}\n"
            f"T1 {x['t1']:.{d}f} | T2
{x['t2']:.{d}f}")
```

```
if __name__ == "__main__":
    res = scan()
    setups = [r for r in res if r["bias"]
!= "WAIT"]
    setups.sort(key=lambda r:
max(r["bull"], r["bear"]) / r["total"],
reverse=True)
    if not setups:
        print("Team ko koi clear consensus
nahi mila.")
    else:
        msg = f"Team scan: {len(setups)}
setup(s) / {len(res)} coins\n\n"
        msg += "\n\n".join(fmt(r) for r in
setups[:MAX_IN_MSG])
        msg += "\n\nSL = 1.5xATR(1h), T1 =
1.5R, T2 = 3R. Advice nahi, SL ke bina
trade mat lo."
        print(msg)
        bot.send_telegram(msg)
```